



BIT1123/BISE2093/DIT1113
OBJECT ORIENTED PROGRAMMING
FINAL PROJECT - GROUP
(40%)

NAME	STUDENT ID	CLASS CODE	PROGRAM	PASSPORT
ALI SHARIF ABDULKADIR SHARIF	(202505010493)	BIT1123	BCSSE	P01572377
Bawazir Bandar Khaled Salem	(202401010116)	BIT1123	BCSSE	10817336
Ahmed Mohammed Fadul Mohammed	(202401010432)	BIT1123	BCSSE	P11003973
Ali Yousef Jalal Abdo	(202401010131)	BIT1123	IT	13107716
ALI ALI ISAK	(202401010128)	BIT1123	IT	P01243861

Declaration of Originality:

I hereby declare that this assignment is my own original work. I have completed this assignment independently and have not copied or plagiarized any part of it from any person, book, website, or other source. Any information, ideas, or references used from external sources have been properly acknowledged where appropriate.

I understand that submitting copied or plagiarized work is a violation of academic integrity and may result in disciplinary action by the university.

GitHub Repository: <https://github.com/bandarbwz/SmartBloodBank>

Table of Contents

1.Project Information & Repository	3
2.1 Introduction & SDG Background	3
2.2 Problem Statement	4
Compatibility errors:	4
Inventory wastage:.....	4
Triage delays:	4
2.3 System Objectives	5
2.4 OOP Design (UML Class Diagram)	6
Model layer	6
Service layer.....	6
UI layer.....	7
Class inventory.....	7
2.5 Implementation Details	8
2.5.1 Encapsulation	8
2.5.2 Inheritance	9
2.5.3 Polymorphism.....	9
2.5.4 Abstraction	10
2.5.5 Collections & File Handling.....	10
2.5.6 Decision-Making & Processing Logic	11
2.6 Testing & Sample Outputs	12
Login.....	12
Dashboard	13
Blood Inventory	14
Emergency Requests	15
Reports	16
Test Scenarios.....	17
2.7 Discussion & Limitations	18
2.8 Conclusion	18
FINAL PROJECT RUBRIC (40%)	19

1. Project Information & Repository

The Smart Blood Bank Management System is a JavaFX desktop application written in Java 21, built with Maven. The complete source code, commit history, and this report's supporting assets are hosted on GitHub at the address below, as required by the assignment brief (Deliverable 1.1).

Repository: <https://github.com/bandarbwz/SmartBloodBank>

The repository follows a package-per-layer structure (model/service / ui) with meaningful class and method names, and Javadoc-style comments on every public class explaining both what the class does and which OOP principle it demonstrates. Demo login credentials for the running application are username admin, password admin123.

2.1 Introduction & SDG Background

Blood banks are a space of emergency and paperwork. In many smaller facilities, finding a compatible unit of blood, checking blood type compatibility, having the correct unit that hasn't expired, and determining who receives the blood first when several requests are placed at the same time still relies on manual processes, usually on paper or spreadsheet. This is a slow way to do it, and as we all know, a hospital can't afford to take a long time when someone's bleeding out.

This project contributes to United Nations Sustainable Development Goal 3: Good Health and Well-being, and to the focus of the goal on strengthening the capacity of health systems to reduce preventable harm. The Smart Blood Bank Management System is a desktop application developed in JavaFX, the new Java-based GUI toolkit that streamlines the basic functions of a blood bank such as patient admission, inventory management, automatic compatibility matching, donor record keeping, and emergency triage. It is not just a repository for records; it is a repository that puts into practice the medical and logistical requirements a real blood bank must meet: For example, it enforces the 42-day shelf-life of a collected blood bag, and the required 90-day interval between whole-blood donations.

The system was intentionally designed as a simulation of actual hospital activities, not just a CRUD (create/read/update/delete) system. Each screen features business logic compatibility checking, FIFO stock rotation and priority queuing which mimics the manual decisions a blood bank coordinator would make.

2.2 Problem Statement

This project addresses three potential failure points with manual or loosely structured blood bank record-keeping.

Compatibility errors:

It is easy to get the blood type wrong when you're busy and it is a transfusion reaction when blood types don't match. The ABO/ Rh compatibility matrix has 8 blood types and the rules governing which ones may be given to which recipients are not symmetrical and can be easily forgotten when it matters most.

Inventory wastage:

If not for the systematic expiry tracking, the blood bags that can be used quietly pass their 42 days of shelf life and are thrown away, further reducing the already limited and difficult-to-replace blood supply. This is worsened when older stock is not prioritised for use ahead of newer stock.

Triage delays:

If more than one patient requires blood simultaneously, there is no way to know which patient will receive the blood first, unless someone is actively operating a priority list on their own. When running a busy ward, that manual task is simple to overlook or to put off.

The system eliminates three problems; the rules are programmed into the system, not on the memory of staff; the rules are applied in the same way every time, not as staff sees fit.

2.3 System Objectives

The following objectives were taken into account when creating the application:

- Keep accurate, up-to-date records of donors and patients, such as donation history and request status.
- Automatically check for blood-type compatibility (default donor/recipient match rules) prior to suggestion or reservation of any match.
- Monitor the availability, reservation, use and expiry date of all blood bags to ensure that they are not used beyond their safe expiry date.
- Handle emergency calls according to priority: Critical, then High, then Medium, then Low with earliest registered first for equal levels of priority.
- Rotate stock on a first in first out basis ensuring the oldest available compatible unit is available before new stock to minimise wastage.
- Display low stock and near expiry items on a dashboard for staff to take action before shortouts.
- Store application data between sessions without having to rely on an external database server.

A secondary goal of the project was to provide a structurally strong design that applied all four of the core OOP principles but that wouldn't function correctly if encapsulation were not respected in the design, and so on.

A secondary goal of the project was to create a structurally strong design that met all four core OOP principles, but that didn't work if one of these principles wasn't respected in the design, and so on. inheritance, polymorphism, or abstraction were removed. Section 2.5 walks through exactly how each principle is used and why.

2.4 OOP Design (UML Class Diagram)

The diagram below (also attached to repository as designs/UML_Class_Diagram.png) illustrates the entire class structure and the relationships between classes.

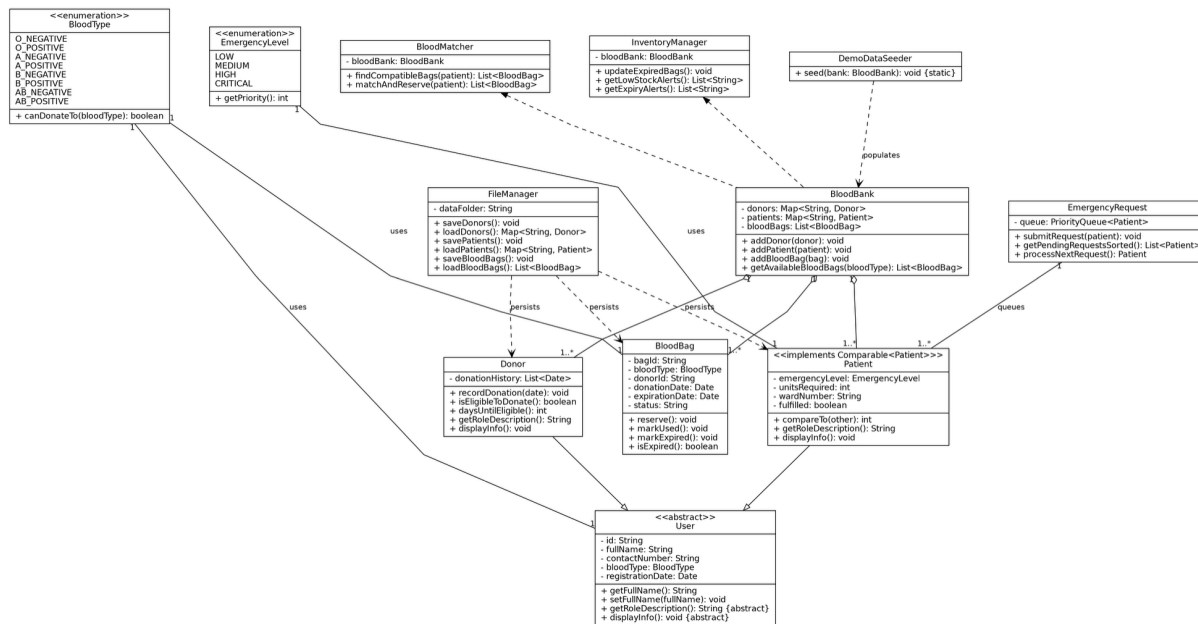


Figure 1 - UML class diagram of the Smart Blood Bank Management System.

The design is organised into three layers.

Model layer

An abstract User class holds everything a person in the system has in common: ID, name, contact number, blood type, and registration date, plus two abstract methods, `getRoleDescription()` and `displayInfo()`, that force each subclass to define its own identity and summary format. Donor and Patient both extend User. Donor adds donation history and eligibility rules; Patient adds emergency level, units required, ward number, and fulfilment status, and additionally implements `Comparable<Patient>` so patient objects can order themselves by urgency without any external sorting logic. Supporting these are BloodBag, an independently encapsulated entity with its own status lifecycle (available → reserved → used/expired), and two enumerations, BloodType (which also carries the donor-compatibility rules) and EmergencyLevel (which carries triage priority).

Service layer

BloodBank, BloodMatcher, EmergencyRequest, InventoryManager, FileManager, and DemoDataSeeder handle business logic. BloodBank is the single in-memory source of truth, keeping donors and patients in HashMaps keyed by ID for fast lookup and blood bags in an ArrayList. BloodMatcher finds compatible, non-expired stock and reserves the soonest-to-expire units first.

EmergencyRequest wraps a PriorityQueue<Patient> so the queue orders itself using Patient's own compareTo(). FileManager handles all persistence to disk.

UI layer

The UI mirrors the same abstract-parent / concrete-child pattern one level up. An abstract Screen class (itself extending JavaFX's BorderPane) defines the shared page chrome: a title bar with the current date, and a scrollable content area, and leaves getTitle() and buildContent() abstract, so each concrete screen only supplies its own title and content.

Class inventory

The application contains 26 classes and enumerations across the three layers, well beyond the assignment's minimum of 6 - 8:

Model	User (abstract)	Shared identity fields and validation for any person in the system
Model	Donor	Donation history, 90-day eligibility rule
Model	Patient	Emergency level, units required, ward, fulfilment status, Comparable ordering
Model	BloodBag	One physical unit of blood; status lifecycle and expiry
Model	BloodType (enum)	8 blood types and donor-to-recipient compatibility rules
Model	EmergencyLevel (enum)	4-level triage priority
Service	BloodBank	In-memory data store for donors, patients, and blood bags
Service	BloodMatcher	Finds and reserves compatible stock, FIFO by expiry
Service	EmergencyRequest	Priority queue of pending patient requests
Service	InventoryManager	Low-stock and near-expiry alerting, expiry sweeps
Service	FileManager	Saves/loads all data to plain-text files
Service	DemoDataSeeder	Populates a fresh install with sample data
UI	MainApp / AppContext / AppShell / ScreenManager	Application bootstrap, shared state, and screen navigation
UI	Screen (abstract)	Shared page chrome for every screen
UI	LoginScreen / DashboardScreen / DonorManagementScreen / PatientManagementScreen / InventoryScreen / EmergencyRequestScreen / ReportsScreen / SettingsScreen	One concrete screen per application feature (8 classes)
UI	ImageResources	Centralised loading of icon/image assets

2.5 Implementation Details

This section provides a walk-through of the implementation of each of the required OOP requirements outlined in the assignment brief, and quotes directly from the source code.

2.5.1 Encapsulation

Every field across the model classes is private. Setters validate their input rather than accepting anything handed to them. `User.setFullName()` and `setContactNumber()` both reject blank input:

```
public void setFullName(String fullName) {
    if (fullName == null || fullName.isBlank()) {
        throw new IllegalArgumentException("Full name cannot be empty.");
    }
    this.fullName = fullName.trim();
}
```

`BloodBag` goes a step further: it exposes no public setter for its status field at all. State can only change through controlled methods that enforce valid transitions, which prevents the object from ever being pushed into an invalid state by outside code:

```
public void reserve() {
    if (status != Status.AVAILABLE) {
        throw new IllegalStateException("Only an AVAILABLE bag can be reserved.");
    }
    status = Status.RESERVED;
}

public void markUsed() {
    if (status != Status.RESERVED) {
        throw new IllegalStateException("Only a RESERVED bag can be marked as used.");
    }
    status = Status.USED;
}
```


2.5.2 Inheritance

User → Donor, Patient in the model layer, Screen → eight concrete screens in the UI layer are two hierarchies that do real structural work, not just to fit into the rubric. In both cases, the parent has true shared state and behaviour and the children only add what is specific to them:

```
public class Donor extends User {
    private final List<LocalDate> donationHistory = new ArrayList<>();

    public Donor(String fullName, String contactNumber, BloodType bloodType) {
        super(fullName, contactNumber, bloodType);
    }
    ...
}

public class Patient extends User implements Comparable<Patient> {
    private EmergencyLevel emergencyLevel;
    private int unitsRequired;
    ...
}
```

2.5.3 Polymorphism

displayInfo() and getRoleDescription() are declared abstract on User and called through User references throughout the reporting code, but the actual output differs at runtime depending on whether the object underneath is a Donor or a Patient — this is runtime polymorphism via superclass references, not just method overriding in isolation:

```
// Donor
@Override
public String displayInfo() {
    return baseInfo() + String.format(" | Total Donations: %d | Eligible Now: %s",
        getTotalDonations(), isEligibleToDonate() ? "Yes" : "No");
}

// Patient
@Override
public String displayInfo() {
    return baseInfo() + String.format(" | Ward: %s | Units Needed: %d | Priority: %s",
        wardNumber, unitsRequired, emergencyLevel.getDescription());
}
```

The same abstract-method-per-subclass pattern repeats in the UI layer, where every Screen subclass supplies its own buildContent() while Screen itself builds the shared header/body frame exactly once.

2.5.4 Abstraction

Note that both User and Screen are abstract classes, which cannot be instantiated directly, only meaningful subclasses can be created. Comparable<Patient> is another kind of abstraction based on interfaces: the emergency queue completely depends on Patient to know how to compare it, and there is no urgency-comparison logic in the queue class itself:

```
@Override
public int compareTo(Patient other) {
    int byUrgency = Integer.compare(
        other.emergencyLevel.getPriority(), this.emergencyLevel.getPriority());
    if (byUrgency != 0) return byUrgency;
    return this.getRegistrationDate().compareTo(other.getRegistrationDate());
}
```

2.5.5 Collections & File Handling

HashMap<String, Donor> and HashMap<String, Patient> back ID lookups in BloodBank; ArrayList holds blood bags and donation history; PriorityQueue<Patient> drives the emergency queue and orders itself automatically using the compareTo() above. FileManager persists all three entity types to pipe-delimited text files under a data/ folder using java.nio.file.Files, and reconstructs objects on startup through dedicated “load” constructors that accept an existing ID and registration date rather than generating new ones:

```
private List<Donor> loadDonors() throws IOException {
    List<Donor> donors = new ArrayList<>();
    for (String line : Files.readAllLines(Paths.get(DONORS_FILE))) {
        String[] fields = line.split(FIELD_SEPARATOR_REGEX, -1);
        Donor donor = new Donor(fields[0], fields[1], fields[2],
            BloodType.valueOf(fields[3]), LocalDate.parse(fields[4]));
        if (fields.length > 5 && !fields[5].isBlank()) {
            for (String date : fields[5].split(LIST_SEPARATOR)) {
                donor.recordDonation(LocalDate.parse(date));
            }
        }
        donors.add(donor);
    }
    return donors;
}
```

2.5.6 Decision-Making & Processing Logic

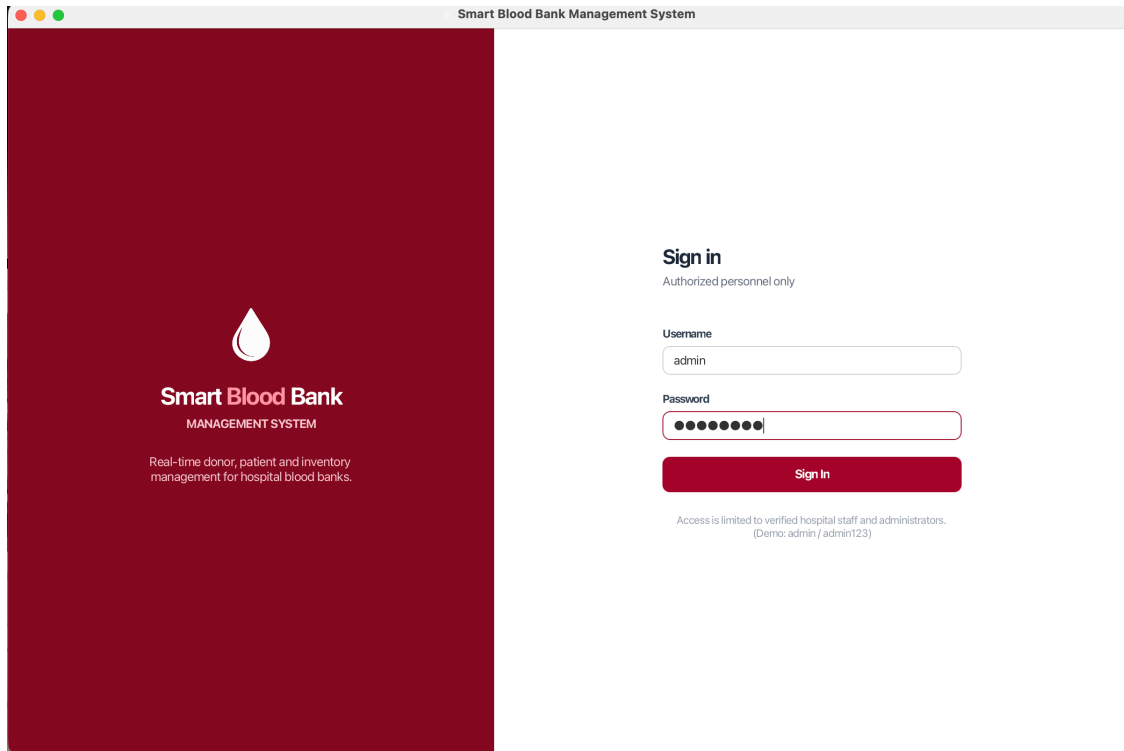
Compatibility between blood types is ensured using switch in `BloodType.canDonateTo()`, representing the eight-type donor/recipient compatibility table, which ensures the implementation of the rule only once in one location rather than being duplicated using if/else statements throughout the codebase. The sorting mechanism implemented in `BloodMatcher` takes advantage of the expiration date of the blood bag candidates to ensure that the blood bags expiring soonest are prioritised for use, and in `processNextRequest()` in `EmergencyRequest`, we check for the availability of blood bags before de-queueing a patient:

```
public Patient processNextRequest() {
    Patient next = pendingRequests.peek();
    if (next == null || !bloodMatcher.canFulfill(next.getBloodType(), next.getUnitsRequired())) {
        return null;
    }
    pendingRequests.poll();
    List<BloodBag> reserved = bloodMatcher.matchAndReserve(next);
    for (BloodBag bag : reserved) bag.markUsed();
    next.markFulfilled();
    return next;
}
```

2.6 Testing & Sample Outputs

The application was tested manually end-to-end using the built-in demo dataset. DemoDataSeeder pre-populates a fresh install with realistic donors, patients, and blood bags, and login uses the demo credentials admin / admin123.

Login



Smart Blood Bank Management System

Sign in
Authorized personnel only

Username
admin

Password
●●●●●●●●

Sign in

Access is limited to verified hospital staff and administrators.
(Demo: admin / admin123)

Figure 2 - Login screen restricting access to authorised staff.

Dashboard

At the time of testing the dashboard correctly reported 15 registered donors, 11 active patient records, 10 available blood units spread across all 8 blood types, and 11 active alerts combining low-stock and near-expiry warnings including a genuine CRITICAL flag on O+ stock, which had 0 units available.

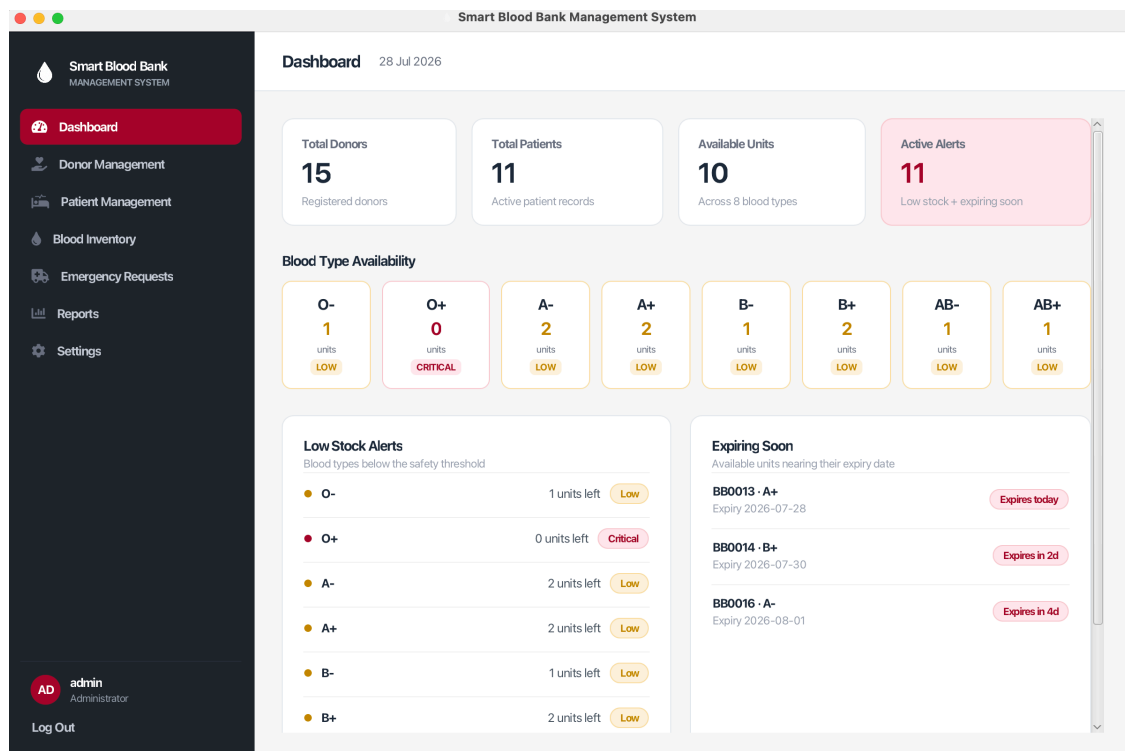


Figure 3 - Dashboard with live stock levels, low-stock alerts, and expiring-soon alerts.

Blood Inventory

The inventory screen lists every blood bag grouped by type, with its collection date, expiry date, countdown to expiry, source donor, and current status. The screenshot below was captured mid-test while adding a new blood bag for donor U1003 (Maria Gonzalez, AB+); it also shows an already-expired O+ bag (BB0006, expired 74 days ago) correctly flagged in red by `InventoryManager.updateExpiredBags()`, confirming the expiry sweep runs and updates status as designed.

The screenshot displays the 'Smart Blood Bank Management System' interface. On the left is a dark sidebar with navigation links: Dashboard, Donor Management, Patient Management, Blood Inventory (highlighted), Emergency Requests, Reports, and Settings. The main content area is titled 'Blood Inventory' with a date '28 Jul 2026'. It shows '18 units across 8 blood types' in a grid of boxes for O-, O+, A-, A+, B-, B+, AB-, and AB+. Below this, the inventory is grouped by type. The 'Type O+' section is expanded, showing a table of blood bags. One bag, BB0006, is marked 'EXPIRED' in red, with a note 'Expired 74d ago'. The 'Add Blood Bag' dialog is open, showing a form to add a new bag for donor 'U1003 — Maria Gonzalez (AB+)' with a donation date of '7/28/2026'. The dialog has 'Cancel' and 'OK' buttons. A '+ Add Blood Bag' button is also visible in the top right of the inventory section.

BAG ID	COLLECTED	EXPIRY	EXPIRES IN	DONOR	STATUS	Rem...
Type O- 1 units						
BB0002	2026-07-22	2026-09-02	Expires in 36d	U1000	AVAILABLE	Rem...
Type O+ 3 units						
BB0006	2026-04-03	2026-05-15	Expired 74d ago	U1004	EXPIRED	Rem...
BB0012	2026-06-17	2026-07-29	Expires in 1d	U1008	USED	Rem...
BB0007	2026-07-09	2026-08-20	Expires in 23d	U1004	USED	Rem...
Type A- 3 units						
BB0008	2026-03-19	2026-04-30	Expired 89d ago	U1005	EXPIRED	Rem...

Figure 4 - Blood inventory grouped by type, with the Add Blood Bag dialog open.

Emergency Requests

The emergency queue correctly sorted patients with 3 requests flagged Life-threatening (Critical) ahead of the remaining 10 pending requests. Selecting patient Grace Adeyemi (A+, 3 units needed, Ward ICU-3) shows the system's suggested compatible bags in FIFO order — BB0013 expiring today, then BB0016 in 4 days, then BB0003 in 31 days, demonstrating that `BloodMatcher.findCompatibleBags()` is genuinely sorting by expiry rather than just returning stock in storage order.

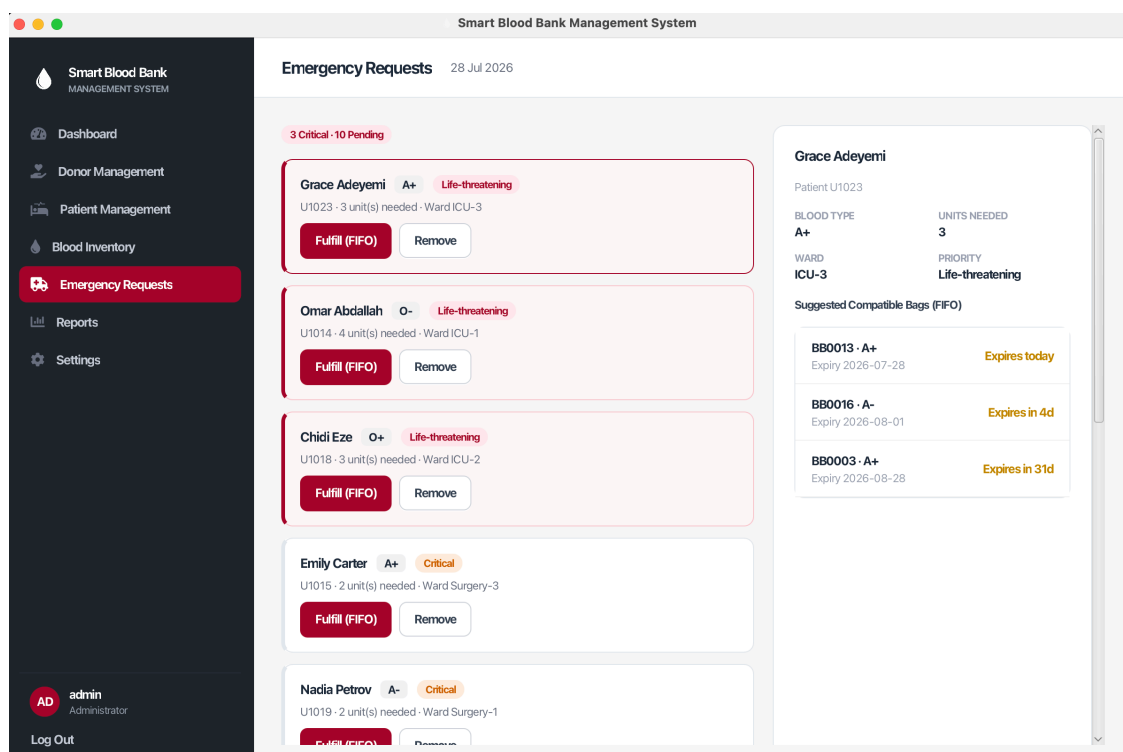


Figure 5 - Emergency request queue with FIFO-ordered compatible bag suggestions.

Reports

The reports screen combines data from BloodBank and FileManager: there are 19 blood donations, 1 of 11 patient requests have been met, 18 blood bags in stock, and 6 blood bags that have expired unused. The monthly donation chart, as well as the inventory bars for each type of blood bag, are calculated from the same donor and blood-bag data displayed in other screens, thus verifying that the reporting layer is not using place-holding information.

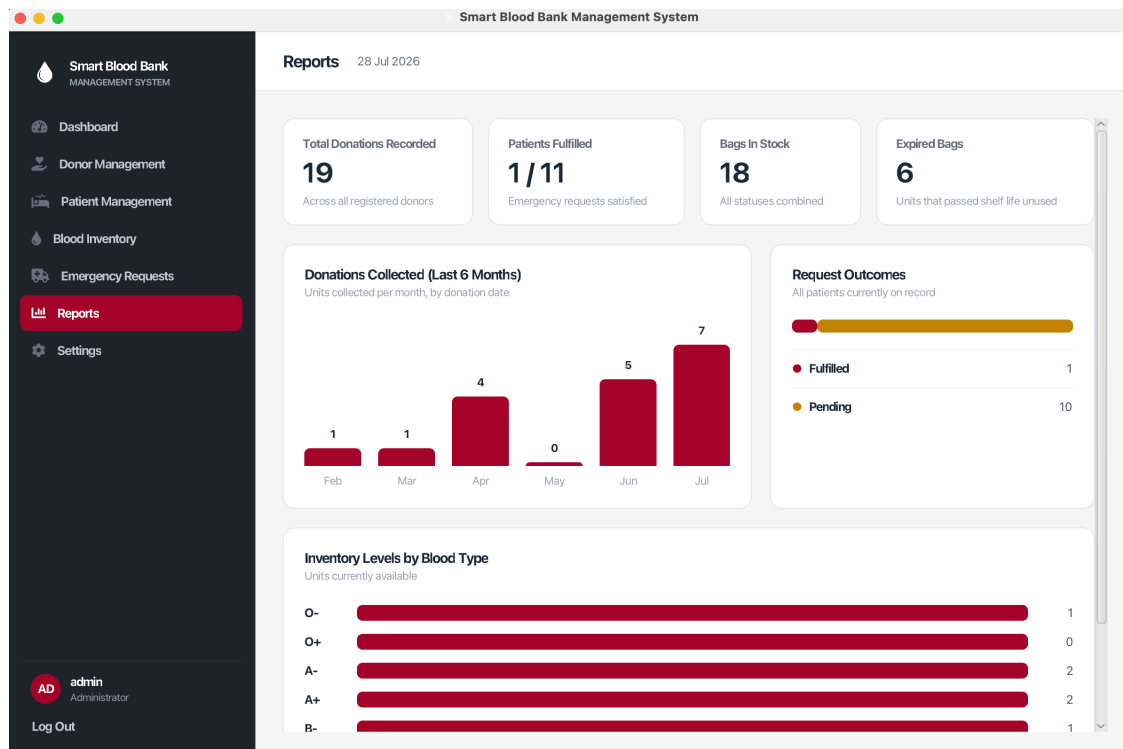


Figure 6 - Reports dashboard showing donation trends, request outcomes, and stock levels by type.

Test Scenarios

In addition to the screens above, the following functional scenarios were manually verified:

Donation eligibility rule	Register a donation for a donor who donated < 90 days ago; expect the system to block it and show days remaining	Pass / blocked correctly
Priority ordering	Submit patients at different urgency levels; expect Critical requests to sit ahead of Low requests regardless of submission order	Pass / queue reorders automatically
Insufficient stock	Request blood for a type with no compatible stock; expect a clear “not enough stock” response instead of a silent failure or wrong match	Pass / <code>IllegalStateException</code> surfaced to UI as a message
Expiry sweep	Add a bag with a donation date > 42 days in the past; expect its status to flip to EXPIRED automatically	Pass / confirmed in Figure 4 (BB0006)
Data persistence	Close and reopen the application; expect all donor, patient, and blood bag data to be restored exactly	Pass / <code>FileManager</code> round-trip verified
Incompatible match prevention	Attempt to match AB+ donor stock to an O- patient; expect it to be excluded from compatible results	Pass / excluded per <code>BloodType.canDonateTo()</code>

2.7 Discussion & Limitations

The system does achieve the goals defined in the marketing materials but it's important to acknowledge its limitations in a realistic manner in comparison to a real hospital deployment.

Data is not stored in a real Data Base but in simple text files. It is fine for a single user desktop demo, but it is not likely to work well if accessed by multiple users, nor would it offer the audit trail or back up features that would be required in a real blood bank.

Unlike a healthy build, there is only one hardcoded logon, so that this version is not able to differentiate between, say, a nurse and an administrator, or between what each of these roles can do.

Here, only the basic ABO and Rh compatibility rules for blood typing are being followed. A production system would also require antibody screening and cross-matching, which would require much more work than a static compatibility table.

This is a simulation of a coursework course and thus doesn't require integration with real hospital information systems, multi terminal network support and no regulatory compliant layer (data protection, retention policy etc.) which would be required by an actual deployment.

None of these are "holes" that I hope are found, but rather are honest limits on what a single-semester, single-application group project can really accomplish. If the project were to be further developed, most of these extensions could be handled without re-writing the code since the architecture is clean with a separation between BloodBank as the data layer and the services that operate on it.

2.8 Conclusion

The Smart Blood Bank Management System aims to demonstrate that the four pillars of OOP are not simply a checklist to be checked for a rubric; but are truly useful when structuring a system with real rules to be enforced like “Who can donate to whom?”, “What is the maximum shelf life of blood?”, and “Who gets treated first in an emergency?” The application remains consistent, easy to extend, and directly addresses SDG 3 by eliminating conditional logic from the user interface and embedding it in the class design itself, an abstract User hierarchy, and a self-guarding BloodBag, instead of the myriad conditional logic spread across the user interface, where both delay and error have an impact on a human life.

FINAL PROJECT RUBRIC (40%)

Section	Description	Weight (Marks)	Very Poor (1)	Poor (2)	Fair (3)	Good (4)	Excellent (5)
SDG Problem Understanding & Relevance	Clarity of SDG selection, problem definition, and real-world relevance	15	SDG unclear or irrelevant; problem poorly defined	SDG mentioned but weak linkage to problem	Adequate SDG linkage; basic problem explanation	Clear SDG alignment; well-defined problem	Strong SDG justification; realistic, impactful problem analysis
System Design & UML	Quality of system architecture and UML class diagram	15	UML missing or incorrect; poor design	Major design errors; incomplete relationships	Basic design; some errors in relationships	Good structure; minor UML mistakes	Excellent, well-structured UML with correct relationships and design logic
Application Functionality	Working features, logic implementation, correctness	20	Application does not run or major failures	Runs but major functional errors	Core features work with some issues	Most features work correctly	Fully functional, stable, logical and realistic simulation
Application of OOP Principles	Implementation of encapsulation, inheritance, polymorphism, abstraction	20	OOP concepts not properly implemented	Limited or incorrect use of OOP	Adequate use of required OOP features	Good integration of OOP concepts	Excellent, meaningful and correct use of all required OOP principles
Use of Collections & File Handling	Data structures and persistence mechanisms	10	No collections or file handling used	Incorrect or minimal implementation	Basic use of collections or file handling	Proper use of collections and file storage	Advanced, well-integrated and efficient data handling
Code Quality & Documentation	Code structure, readability, comments, naming conventions	10	Very poor structure; unreadable	Poor formatting; minimal comments	Acceptable structure and readability	Good organization and documentation	Very clean, modular, professional-quality code
Report Quality	Clarity, structure, explanation, discussion	5	Very poorly structured; incomplete	Weak explanations; unclear sections	Adequate explanation and structure	Well-written and organized	Professional, clear, critical discussion included
Presentation & Demonstration	Clarity of explanation, demo quality, teamwork	5	Unclear presentation; unable to demo	Weak demo; limited explanation	Basic demo and explanation	Clear explanation and smooth demo	Confident, professional presentation; strong engagement
Team Contribution & Collaboration	Evidence of teamwork and equal contribution	5	Major imbalance in contribution	Uneven contribution	Mostly balanced contribution	Good collaboration	Excellent teamwork; clear equal contribution



BIT1123 · OBJECT ORIENTED PROGRAMMING · FINAL GROUP PROJECT (40%)



Smart Blood Bank Management System

A JavaFX OOP Simulation Supporting UN SDG 3 Good Health & Well-being

TEAM : CLASS BIT1123

Ali Sharif Abdulkadir Sharif / 202505010493

Bawazir Bandar Khaled Salem / 202401010116

Ahmed Mohammed Fadul Mohammed / 202401010432

Ali Yousef Jalal Abdo / 202401010131

Ali Ali Isak / 202401010128

SDG 3 · GOOD HEALTH & WELL-BEING

Problem Statement & Objectives

Strengthening health-system capacity to reduce preventable harm from blood-supply errors and delays.



Compatibility errors

8 blood types, asymmetric ABO/Rh rules easy to mismatch under pressure.



Inventory wastage

42-day shelf life bags expire unused when stock isn't rotated FIFO.



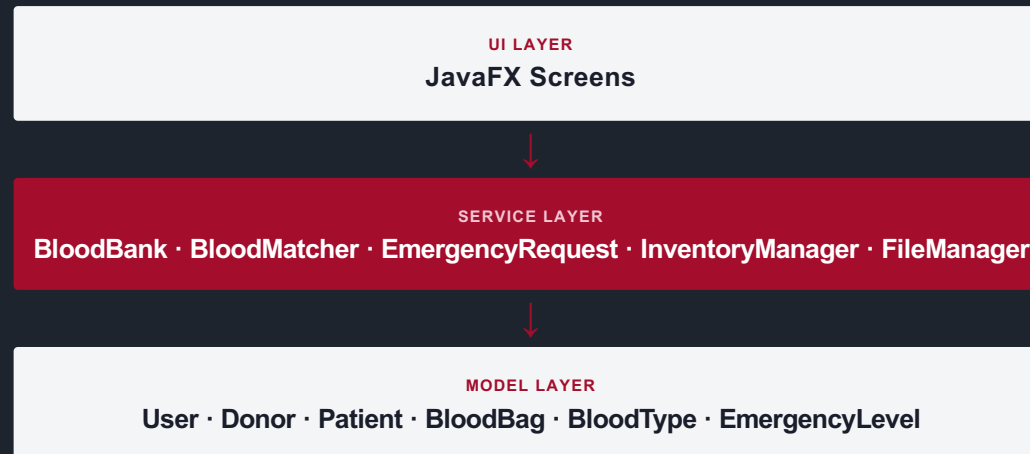
Triage delays

No automatic way to know which patient is served first when requests collide.

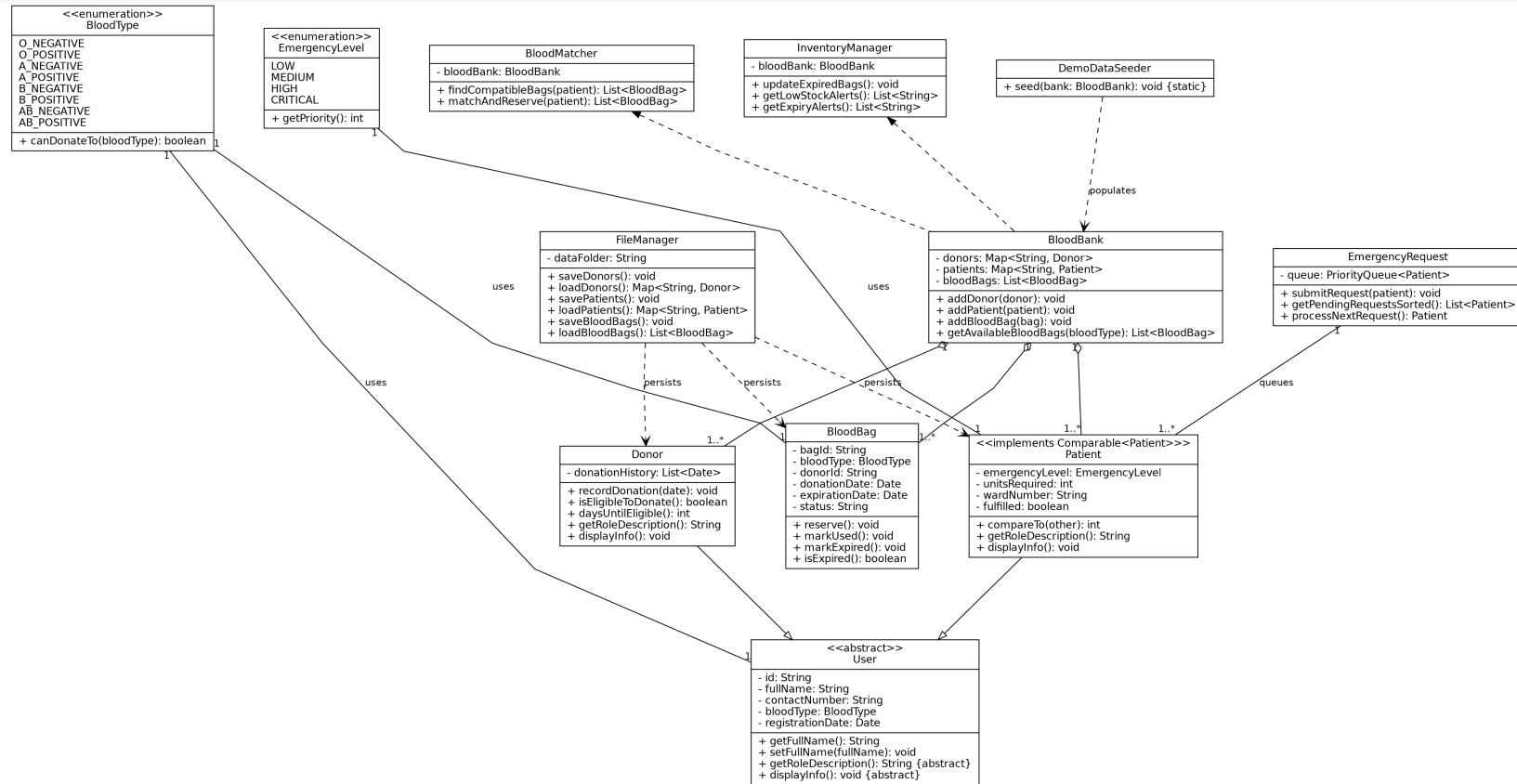
OBJECTIVES

- Automatic compatibility checking before any match
- Priority emergency queue (Critical → Low)
- FIFO stock rotation by expiry date
- Durable file-based persistence, no external DB

System Architecture



UML Class Diagram



User → Donor / Patient inheritance · 26-class structure across 3 layers

How the App Works

One end-to-end user journey through the system

1

Staff Login

Authenticate on LoginScreen before any record can be viewed or changed.

2

Register Donor / Patient

Validated input stored in BloodBank's donor and patient maps.

3

Submit Emergency Request

Patient enters the priority queue, ordered by EmergencyLevel.

4

Auto-Match Compatible Stock

BloodMatcher finds compatible, non-expired bags oldest expiry first.

5

Reserve & Fulfill

Bags move reserved → used; the patient is marked fulfilled.

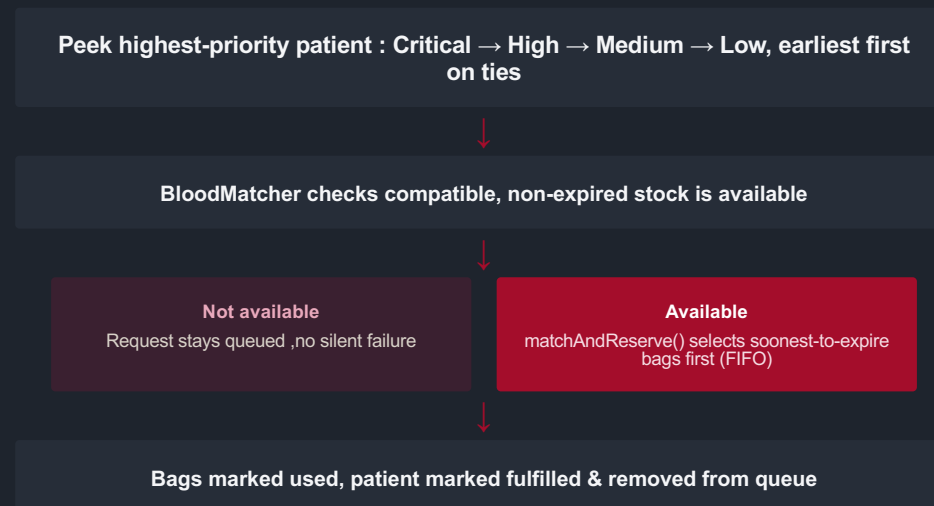
6

Dashboard & Reports Update

InventoryManager refreshes low-stock and expiry alerts in real time.

Matching & Priority Logic

Inside EmergencyRequest.processNextRequest()



OOP Principles in Practice

Encapsulation

Private fields + validated setters. `BloodBag` guards its own status transitions, no public setter for status.

Inheritance

`User` → `Donor`, `Patient` and `Screen` → 8 concrete screens, both with real shared behavior.

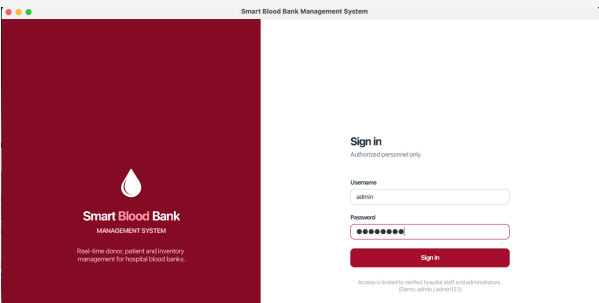
Polymorphism

Overridden `displayInfo()` / `getRoleDescription()`, dispatched through plain `User` references.

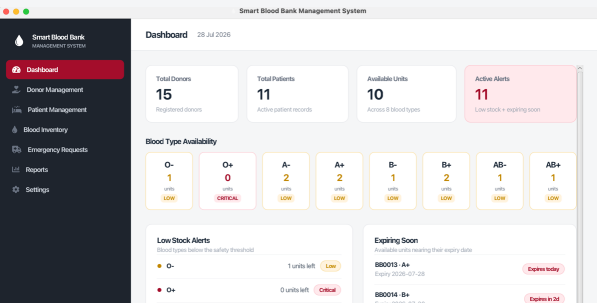
Abstraction

`AbstractUser` / `Screen`, plus `Patient` implements `Comparable<Patient>`.

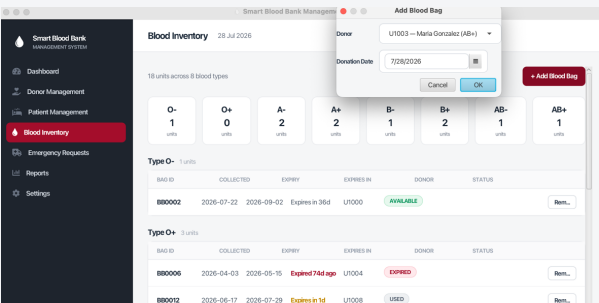
Application Screenshots



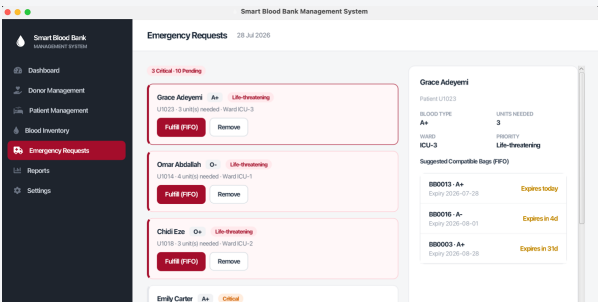
Login screen



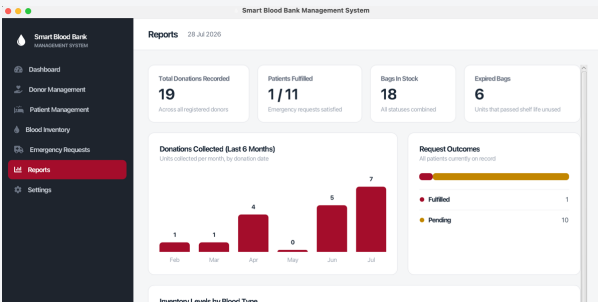
Live dashboard with stock levels & alerts



Blood inventory grouped by type



Emergency queue, FIFO-ordered compatible bags



Reports dashboard

Testing, Limitations & Conclusion

TEST SCENARIOS

SCENARIO	RESULT
Priority queue reordering	Pass
Expiry sweep (42-day rule)	Pass
Incompatible match blocked	Pass

LIMITATIONS

- Flat-file storage, no real database
- Single hardcoded login, no role-based access

Clean OOP design turns life-critical rules into enforced code, not UI conditionals , directly advancing SDG 3.



Smart Blood Bank
Management System

Thank You